JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Interface AsynchronousChannel

All Superinterfaces:

`AutoCloseable`, `Channel`, `Closeable`

All Known Subinterfaces:

`AsynchronousByteChannel`

All Known Implementing Classes:

`AsynchronousFileChannel`, `AsynchronousServerSocketChannel`, `AsynchronousSocketChannel`

```
public interface AsynchronousChannel
extends Channel
```

A channel that supports asynchronous I/O operations. Asynchronous I/O operations will usually take one of two forms:

1. `Future<V> operation(...)`
2. `void operation(... A attachment, CompletionHandler<V,? super A> handler)`

where *operation* is the name of the I/O operation (read or write for example), *V* is the result type of the I/O operation, and *A* is the type of an object attached to the I/O operation to provide context when consuming the result. The attachment is important for cases where a *state-less* `CompletionHandler` is used to consume the result of many I/O operations.

In the first form, the methods defined by the `Future` interface may be used to check if the operation has completed, wait for its completion, and to retrieve the result. In the second form, a `CompletionHandler` is invoked to consume the result of the I/O operation when it completes or fails.

A channel that implements this interface is *asynchronously closeable*: If an I/O operation is outstanding on the channel and the channel's `close` method is

invoked, then the I/O operation fails with the exception `AsynchronousCloseException`.

Asynchronous channels are safe for use by multiple concurrent threads. Some channel implementations may support concurrent reading and writing, but may not allow more than one read and one write operation to be outstanding at any given time.

Cancellation

The `Future` interface defines the `cancel` method to cancel execution. This causes all threads waiting on the result of the I/O operation to throw `CancellationException`. Whether the underlying I/O operation can be cancelled is highly implementation specific and therefore not specified. Where cancellation leaves the channel, or the entity to which it is connected, in an inconsistent state, then the channel is put into an implementation specific *error state* that prevents further attempts to initiate I/O operations that are *similar* to the operation that was cancelled. For example, if a read operation is cancelled but the implementation cannot guarantee that bytes have not been read from the channel then it puts the channel into an error state; further attempts to initiate a `read` operation cause an unspecified runtime exception to be thrown. Similarly, if a write operation is cancelled but the implementation cannot guarantee that bytes have not been written to the channel then subsequent attempts to initiate a `write` will fail with an unspecified runtime exception.

Where the `cancel` method is invoked with the `mayInterruptIfRunning` parameter set to `true` then the I/O operation may be interrupted by closing the channel. In that case all threads waiting on the result of the I/O operation throw `CancellationException` and any other I/O operations outstanding on the channel complete with the exception `AsynchronousCloseException`.

Where the `cancel` method is invoked to cancel read or write operations then it is recommended that all buffers used in the I/O operations be discarded or care taken to ensure that the buffers are not accessed while the channel remains open.

Since:

1.7

Method Summary

		Abstract Methods
Modifier and Type	**Method**	**Description**
void	**close**()	Closes this channel.

Methods inherited from interface java.nio.channels.Channel

isOpen

Method Detail

close

```
void close()
    throws IOException
```

Closes this channel.

Any outstanding asynchronous operations upon this channel will complete with the exception AsynchronousCloseException. After a channel is closed, further attempts to initiate asynchronous I/O operations complete immediately with cause ClosedChannelException.

This method otherwise behaves exactly as specified by the Channel interface.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Channel

Specified by:

close in interface Closeable

Throws:

C

`IOException` - If an I/O error occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.